Semantic MapReduce: Orchestrating LLMs over Large-Scale Data

Anonymous Author(s)

Abstract

Large-scale data systems efficiently scan, join, and aggregate massive datasets, but they do not directly solve semantic interpretation tasks such as evidence fusion, incident hypothesis generation, explanation, and auditable reasoning. Directly attaching an LLM to a database, log store, or dashboard is also insufficient: context is too large, intermediate evidence is hard to audit, and global conclusions must be derived from partitioned observations. This paper studies how an LLM-augmented system should orchestrate large-scale analysis when the hard part is not data access but semantic reduction. We frame the core abstraction as *Semantic MapReduce*: a small operator vocabulary for sharding observations, extracting local evidence, normalizing and grouping evidence objects, semantically reducing them into hypotheses, and generating traceable reports. We instantiate this abstraction in an anonymized orchestration-layer prototype that turns LLM reasoning workflows into explicit dataflow/stream pipelines, keeps model serving and data engines behind integration contracts, and makes evidence objects and workflow traces first-class artifacts. We introduce a reproducible large-scale analysis workload based on NPU-backed LLM serving telemetry. Across a small multi-seed matrix, its deterministic incident reducer reaches mean precision 0.9333, mean recall 0.9583, and mean F1 0.9392, compared with 0.2250 mean F1 for a map-only alert baseline and 0.7600 for a window-aggregation baseline.

1 Introduction

Modern data systems are very good at moving, filtering, joining, and aggregating data. Hadoop MapReduce made large-scale analysis programmable through local map tasks followed by global reduction [2]; Spark and Flink improved the execution model for iterative, in-memory, batch, and stream workloads [1, 8]; Ray generalized distributed execution for emerging AI applications [6]. These systems are essential infrastructure, but they do not directly answer a different class of analysis questions: are these distributed symptoms part of the same incident, what evidence supports the hypothesis, which evidence conflicts with it, and what action should an operator take next?

This distinction appears even in a controlled telemetry workload. In an NPU-backed LLM serving scenario, Figure 1 shows that map-only alerts reach 0.2250 mean F1 and window aggregation reaches 0.7600, while an incident-level reducer reaches 0.9392. The reason is not faster scanning; it is

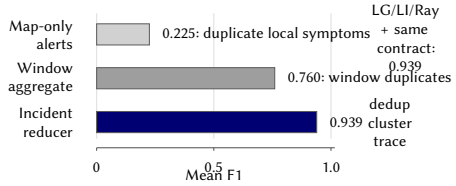

Figure 1. Motivating result. The advantage here comes from the incident reducer contract: it deduplicates local/window evidence into traceable hypotheses. Adjacent adapters match only when reusing that contract.

that the reducer turns duplicate shard symptoms and window-level detections into evidence-linked incident hypotheses. A LangGraph/LlamaIndex/Ray-local adapter probe matches this number only when supplied with the same evidence contract, reducer, and scorer. Existing substrates can carry the computation; the missing system abstraction is first-class semantic reduction with auditable evidence.

Large language models (LLMs) make this semantic layer newly programmable. An LLM can summarize logs, classify symptoms, explain anomalous metrics, and produce operator-facing narratives. However, a simple "LLM over the database" architecture is a poor systems boundary. The input may contain millions of events, traces, metrics, documents, and experiment outputs. Relevant observations are partitioned by service, tenant, region, time window, and storage system. The output must be auditable: a conclusion should link back to concrete evidence objects rather than only to a transient prompt. Finally, cost, latency, privacy, and safety constraints make it undesirable to send all raw telemetry to a general-purpose model.

We therefore study the following systems problem: *how to orchestrate LLM-augmented large-scale analysis when the central operation is reducing distributed evidence into traceable semantic conclusions.*

We argue that the right abstraction is neither a monolithic agent nor a replacement for existing data engines. It preserves the local/global structure that made MapReduce effective, while changing the meaning of reduction. In traditional analytics, reduce computes values such as sums, counts, maxima, joins, and group-by aggregates. In LLM-native analysis, reduce combines evidence, deduplicates related symptoms, resolves conflicts, generates hypotheses, and produces explanations that remain linked to the observations that support them. We call this abstraction *Semantic MapReduce*, and

define it around standard operators rather than around a single prompt or application script.

We instantiate this idea in an anonymized prototype. The prototype is a dataflow-native framework for modular, controllable, and transparent LLM-augmented reasoning. Its stream API exposes familiar dataflow composition primitives, its runtime provides local and optional distributed execution entrypoints, and its serving integration keeps inference engines outside the core package through explicit contracts. Rather than treating LLM analysis as a single prompt-response interaction, the prototype represents analysis as a workflow graph whose operators can be inspected, replayed, replaced, and connected to external systems. This makes the prototype an orchestration layer above or beside Spark, Flink, Ray, databases, vector systems, and LLM serving engines.

The resulting capability is stronger than a chat interface over telemetry. The prototype can express a pipeline in which different shards are analyzed independently, local outputs are normalized into evidence objects, reduction logic is selected by policy, and final explanations remain connected to the intermediate decisions that produced them. The same workflow can run with a deterministic reducer, with a stub reducer for interface testing, or with an LLM-backed reducer for semantic fusion. The prototype therefore makes LLM-native analysis operationally structured.

The evidence boundary is deliberately narrow. The current workload does not implement production-grade distributed shuffle, exactly-once fault tolerance, or a real LLM reducer. It implements a reproducible workload and a deterministic reducer baseline, plus an `llm-stub` reducer that fixes the interface for LLM-backed reduction. The supported claim is that the prototype expresses MapReduce-like semantic orchestration as a standard operator workflow and exposes meaningful reducer failure modes. It is not a replacement for existing execution engines.

This paper makes five contributions:

1. It defines Semantic MapReduce as a systems abstraction and operator vocabulary for LLM-native analysis over partitioned data.
2. It maps the abstraction onto workflow, stream, runtime, evidence, and serving-integration boundaries, emphasizing how an orchestration layer can sit above existing data and execution engines.
3. It describes the mechanisms that make the abstraction practical: explicit workflow graphs, stream composition, reducer plug-ins, evidence objects, and auditable traces.
4. It introduces a reproducible large-scale analysis workload for NPU-backed LLM serving telemetry, including injected incidents, evidence operators, incident reduction, and precision/recall/F1 scoring.
5. It reports a small multi-seed evaluation against diagnostic map-only and window-aggregation baselines, showing that incident-level semantic reduction improves F1 while leaving concrete failure modes for LLM-backed semantic reducers.

2 Motivation

What analytics engines leave to humans. Operational analysis often starts with numerical aggregates but ends with semantic judgment. Consider an LLM serving platform backed by accelerators. A dashboard may show that p95 latency increased in one region, decode utilization saturated, scheduler queue depth grew, and errors rose for some tenants. Each observation is measurable with conventional analytics. The harder question is whether these observations are one incident, several unrelated incidents, or noise. An operator must connect symptoms across services, identify the likely bottleneck, discount misleading signals, and decide what to inspect next.

Traditional execution engines provide the substrate for scans, windows, joins, and stateful aggregation. They do not normally provide a first-class abstraction for incident hypotheses, evidence objects, conflict handling, or explanation traces. These semantic tasks are often implemented as notebooks, scripts, alert rules, dashboards, and human reasoning. The result is powerful but hard to reproduce: two engineers may examine the same data and reach different conclusions, and the reasoning path may not survive the debugging session.

Why direct LLM integration is not enough. LLM-enabled data tools often expose a chat interface over a database, log store, vector index, or dashboard. This interface is useful for ad hoc exploration, but it hides important systems structure. The model must decide which context to inspect, how much evidence to compress, when to issue follow-up queries, and how to justify its answer. Without an explicit workflow, the analysis is difficult to schedule, parallelize, cache, replay, and audit.

Large-scale analysis needs a more structured design. First, raw data must be partitioned so each analysis unit fits within compute and context limits. Second, local outputs must be normalized into evidence objects with provenance. Third, reduction must be explicit, because it is the stage where duplicate evidence, contradictory signals, and weak hypotheses are accepted or rejected. Fourth, final reports must preserve traceability, especially if they are used in operational settings.

Why an orchestration layer is the right layer. The orchestration layer does not replace Spark, Flink, Ray, SQL engines, vector stores, or serving engines. Those systems already own important lower-level responsibilities: distributed execution, storage access, stream processing, indexing, model serving, and resource management. The orchestration layer instead expresses the LLM reasoning workflow as a dataflow/stream

pipeline, coordinates semantic operators, and records intermediate evidence. This layer is where sharding, evidence extraction, grouping, semantic reduction, and explanation generation become explicit.

3 Problem Statement

We define large-scale LLM-augmented data analysis as follows. The input is a large collection of heterogeneous observations: events, logs, metrics, traces, documents, experiment outputs, and derived aggregates. The output is a set of structured insights, incident hypotheses, explanations, evidence chains, and possible actions. A correct system must manage both scale and semantics.

This problem exposes six system challenges:

- **Scale and partitioning.** Data must be split across shards while preserving enough local context to extract meaningful evidence.
- **Evidence compression.** Raw records cannot all be sent to a reducer or LLM. Map stages must summarize without losing the signals needed for global reasoning.
- **Semantic reduction.** The reducer must merge related symptoms, remove duplicates, handle conflicts, and produce hypotheses rather than only numeric aggregates.
- **Auditability.** Reports must link back to the evidence and workflow decisions that produced them.
- **Latency and cost.** LLM calls, if used, must be bounded and placed where semantic value justifies cost.
- **Safety boundaries.** The orchestration layer must avoid leaking unnecessary raw data and must make it possible to restrict what reaches external models.

These constraints motivate a system that treats LLM interpretation as a workflow over evidence objects, not as a final chat step after data processing.

4 Semantic MapReduce

The central claim of Semantic MapReduce is that LLM-native analysis needs standard operators, analogous in spirit to `map` and `reduce`, but with contracts for evidence and explanation. We use six operators throughout the paper: Shard partitions observations by scale, locality, policy, or context budget; MapEvidence extracts bounded local evidence; Normalize converts local outputs into ranked evidence objects; GroupEvidence groups evidence by incident key, service, region, time, or semantic similarity; SemanticReduce merges, deduplicates, adjudicates conflicts, and emits hypotheses; and ReportTrace generates evidence-linked reports with operator provenance. External data engines may implement the scale-facing operators, while LLM frameworks may implement semantic reduction or reporting. The orchestration layer composes them under one auditable evidence contract.

Figure 2 summarizes the abstraction. The six operators form three logical stages.

Algorithm 1 Semantic MapReduce Workflow

Require: Observations D, operators S, M_E, N, G_E, R_S, G_T
Ensure: Evidence-linked incident report $\mathcal{Y}$ and workflow trace $\mathcal{T}$

$\mathcal{S} \leftarrow S(D)$ ▷ Shard by service, region, tenant, or time
$\mathcal{E} \leftarrow \emptyset, \mathcal{T} \leftarrow \emptyset$
for all shard $s \in \mathcal{S}$ **do**
  $(E_s, T_s) \leftarrow M_E(s)$ ▷ MapEvidence
  $\mathcal{E} \leftarrow \mathcal{E} \cup E_s$
  $\mathcal{T} \leftarrow \mathcal{T} \cup T_s$
end for
$\mathcal{E}' \leftarrow N(\mathcal{E})$ ▷ Normalize
$\mathcal{C} \leftarrow G_E(\mathcal{E}')$ ▷ GroupEvidence
$(\mathcal{H}, T_R) \leftarrow R_S(\mathcal{C})$ ▷ SemanticReduce
$\mathcal{T} \leftarrow \mathcal{T} \cup T_R$
$\mathcal{Y} \leftarrow G_T(\mathcal{H}, \mathcal{E}', \mathcal{T})$ ▷ ReportTrace
return $(\mathcal{Y}, \mathcal{T})$

Shard and map evidence. Each shard is processed locally. MapEvidence may compute statistics, identify anomalies, summarize logs, classify symptoms, or invoke an LLM over a bounded context. Its output is not arbitrary prose. It is an evidence object with fields such as service, tenant, region, metric, time window, observed value, baseline, severity, confidence, and source references.

Normalize, group, and semantic reduce. Normalize and GroupEvidence prepare evidence for global reasoning. SemanticReduce then merges evidence objects into higher-level hypotheses. This stage deduplicates adjacent windows, combines weak but consistent signals, separates unrelated incidents, handles contradictory evidence, and assigns confidence. The reducer may be deterministic, LLM-backed, or hybrid. Deterministic reducers are useful baselines because they are reproducible and cheap; LLM reducers are attractive when the merge requires domain language, causal reasoning, or flexible explanation.

Report and trace. ReportTrace turns hypotheses into operator-facing reports. The report should include the claim, supporting evidence, possible conflicting evidence, and suggested actions. It should also preserve the workflow trace so a user can inspect how the conclusion was formed.

Semantic MapReduce differs from an agent loop because its units of parallelism, evidence compression, reduction, and reporting are explicit. It also differs from ordinary stream processing: stream windows can compute features and alerts, but semantic reduction turns local findings into a global explanation.

Algorithm 1 makes the operator vocabulary operational. The important constraint is that SemanticReduce consumes compact, normalized, grouped evidence objects rather than raw telemetry. This is what lets the orchestration layer place expensive or policy-sensitive LLM calls at semantic boundaries instead of at every scan or filter operation.

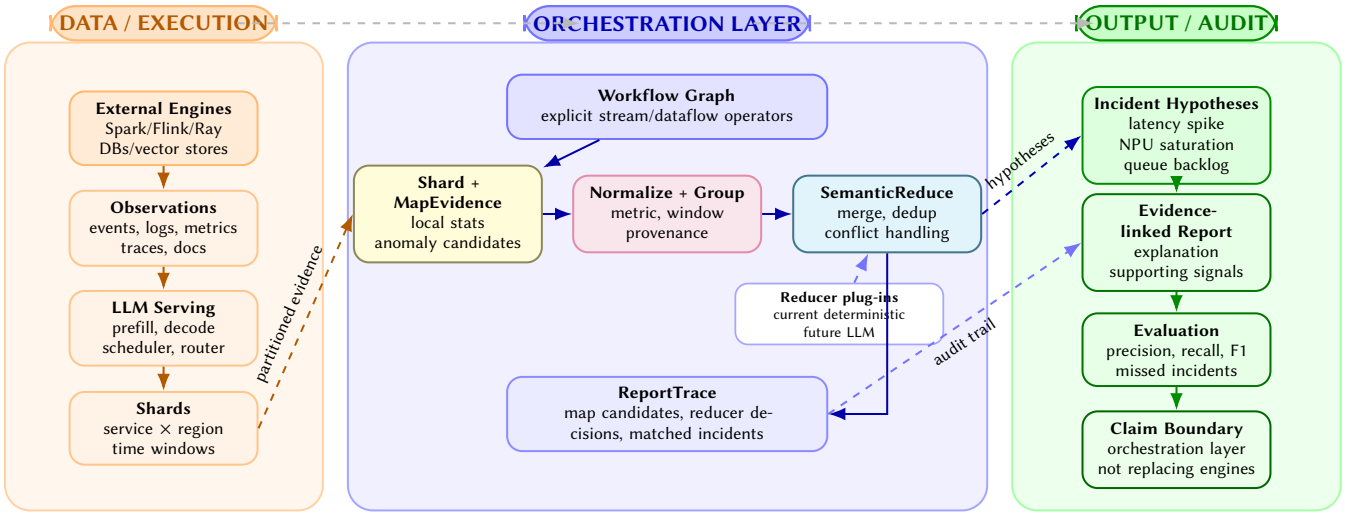

Figure 2. Semantic MapReduce as an orchestration layer over existing data and execution systems. External engines produce partitioned observations; the orchestration layer coordinates Shard, MapEvidence, Normalize, GroupEvidence, SemanticReduce, and ReportTrace; the output is a set of evidence-linked incident hypotheses and evaluation artifacts. Solid arrows show the main data path, while dashed arrows show cross-boundary handoffs and audit links.

5 System Design

5.1 Design Goals and Non-goals

The prototype targets four design goals for Semantic MapReduce.

Explicit workflow. Analysis is represented as a graph of operators so that task decomposition, intermediate evidence, and final reports are visible.

Composable streams. The system reuses stream/dataflow idioms for map, filter, flatmap, key grouping, and operator connection.

Pluggable semantic operators. The map and reduce stages support deterministic logic, LLM-backed logic, or hybrids under the same evidence schema.

Auditable evidence. The reducer produces structured incidents that reference the evidence used to form them.

The prototype also has explicit non-goals. It does not implement a full distributed shuffle for this workload. It does not provide exactly-once fault tolerance for this workload. It does not replace external data engines or model serving runtimes. These boundaries keep the current contribution focused on orchestration and workload design.

5.2 System Boundary and Workflow Model

Figure 2 also shows the intended system boundary. External systems provide raw observations, pre-aggregation, indexing, and model serving. The orchestration layer coordinates the semantic workflow: Shard, MapEvidence, Normalize, GroupEvidence, SemanticReduce, and ReportTrace.

This boundary is important for comparison. Spark, Flink, and Ray can execute large computations; an orchestration layer can call or sit beside them when semantic orchestration is needed. LlamaIndex, LangGraph, LangChain, and AutoGen provide useful LLM application abstractions, but they are not primarily dataflow runtimes for auditable large-scale evidence reduction. Data+AI platforms provide deep integration, but this design emphasizes an open workflow/runtime boundary and explicit evidence trace.

***Workflow graph as the operator IR.*.** The workflow graph serves as an intermediate representation for LLM-augmented reasoning. A direct LLM integration often leaves the reasoning path inside a prompt chain or application glue code. In contrast, a workflow decomposes analysis into the standard operators introduced above. MapEvidence has a different role from Normalize; SemanticReduce has a different role from ReportTrace. This separation gives the system several practical properties.

First, the workflow can be inspected. A developer can see where evidence is extracted, where it is compressed, and where global hypotheses are formed. Second, the workflow can be replayed. If a reducer emits a wrong incident, the system can rerun the same map outputs through a new reducer instead of regenerating all raw telemetry. Third, operators can be replaced independently. A deterministic reducer can be used for CI and regression tests, while an LLM-backed reducer can be evaluated under the same schema. Fourth, the workflow graph gives the runtime a place to record provenance: each final incident can be connected to the map outputs and reducer decision that produced it.

This is the sense in which the system is more than a prompt wrapper. The graph is not only a programming convenience;

Table 1. Division of responsibility in the Semantic MapReduce stack. The prototype is positioned as the orchestration layer rather than a replacement for data engines or model-serving systems.

Layer	Primary responsibility	Why it matters for LLM-native analysis
Data engines	Storage access, scans, joins, stream windows, pre-aggregation, distributed execution	Keeps high-volume data movement in systems already optimized for it; avoids making the orchestration layer a replacement for Spark, Flink, Ray, or databases.
LLM serving engines	Model execution, batching, scheduling, tokenizer/model-specific serving policy	Keeps model serving modular; a semantic reducer can call external models without embedding a serving engine in the workflow layer.
Workflow/stream runtime	Operator graph, stream composition, map/reduce orchestration, reducer selection, trace capture	Makes LLM reasoning explicit, replayable, and inspectable instead of hiding it in a prompt chain.
Semantic operators	Evidence extraction, hypothesis merging, conflict handling, explanation generation	Encodes the semantic work that conventional aggregations do not express: merging symptoms into auditable conclusions.
Evaluation harness	Ground truth, precision/recall/F1, missed incidents, reducer comparisons	Provides a reproducible way to compare deterministic, LLM-backed, and hybrid reducers on identical workloads.

it is the unit of reproducibility and auditability for semantic analysis.

Stream composition as the data-system interface. The second advantage is that the system expresses LLM reasoning through dataflow and stream composition rather than through one-off callbacks. The stream API exposes operators such as map, filter, flatmap, key grouping, and stream connection. These primitives are familiar from data systems, but here they instantiate the Semantic MapReduce vocabulary: sharded map stages implement MapEvidence, keyed stages implement GroupEvidence, and downstream stages invoke SemanticReduce and ReportTrace.

This stream boundary also gives the system a clean integration path. Upstream systems can provide already-partitioned events, pre-aggregated windows, or document chunks. The orchestration layer does not need to own the storage engine to coordinate the semantic workflow. Downstream systems can consume incident objects, reports, or traces. The system is therefore complementary: it adds a semantic orchestration plane without owning every execution primitive below it.

Evidence objects as the semantic data model. The workload uses structured evidence rather than free-form text. A simplified evidence object contains:

- shard and time-window identifiers;
- service, region, tenant, and metric fields;
- observed values and local baseline statistics;
- anomaly type and severity;
- confidence and source-event references.

This schema is intentionally model-agnostic. A deterministic mapper can populate it from thresholds and percentiles. A future LLM mapper can populate it from logs or traces. A reducer can then operate over the same evidence representation. The key design choice is that evidence is not merely embedded in prose. It is a first-class object that can be scored, matched to ground truth, serialized in reports, and inspected after the workflow completes.

Evidence objects are the bridge between data systems and LLM reasoning. Data engines usually produce rows, windows, and aggregates. LLMs often produce text. Semantic MapReduce needs an intermediate form that is structured enough for systems evaluation and expressive enough for semantic interpretation. The evidence object plays that role.

5.3 Reducers, Traces, and Integration

The current code introduces a reducer abstraction with four concrete roles: a map-only diagnostic baseline, a window-aggregation baseline, a deterministic incident reducer, and an `llm-stub` reducer. The map-only baseline reports shard-local candidates directly. The window-aggregation baseline merges candidates within each service/region/window but does not cluster adjacent windows into incident-level hypotheses. The deterministic reducer performs that incident-level clustering using structured fields. The stub reducer preserves the future LLM interface while delegating to the deterministic baseline so CI and offline experiments remain reproducible.

The reducer interface is the extension point for LLM-backed reduction. An LLM reducer can take evidence objects, call a model under bounded context, and emit the same incident schema. Because scoring is unchanged, deterministic and LLM reducers can be compared on identical seeds and ground truth. This is an important systems property: changing the semantic reducer does not require rewriting the generator, shard mapper, scorer, report schema, or benchmark matrix.

Algorithm 2 Reducer Contract Used by the Workload

Require: Ranked evidence objects $\mathcal{E}$, reducer policy π, evidence budget k
Ensure: Incident hypotheses $\mathcal{H}$ and reducer trace T_R
$C \leftarrow \textsc{GroupCandidates}(\mathcal{E}, \text{type, service, region, window})$
$\mathcal{H} \leftarrow \emptyset, T_R \leftarrow \emptyset$
for all candidate group $c \in C$ **do**
  $E_c \leftarrow \textsc{SelectTopEvidence}(c, k)$
  if $\pi = \textsc{Deterministic}$ **then**
    $h \leftarrow \textsc{RuleMerge}(E_c)$
  else if $\pi = \text{LLM}$ **then**
    $h \leftarrow \text{LLMMerge}(E_c)$ ▷ Future implementation
  else
    $h \leftarrow \textsc{HybridMerge}(E_c)$ ▷ Rules filter, LLM adjudicates
  end if
  if $\textsc{IsValidIncident}(h)$ **then**
    $\mathcal{H} \leftarrow \mathcal{H} \cup \{h\}$
    $T_R \leftarrow T_R \cup \textsc{TraceDecision}(h, E_c, \pi)$
  end if
end for
return $(\mathcal{H}, T_R)$

Algorithm 2 shows the reducer interface used to separate evidence production from hypothesis formation. The current implementation includes diagnostic non-semantic reducers as well as the deterministic incident reducer; the `llm-stub` branch preserves the LLM call boundary. An LLM implementation changes how evidence is merged into hypotheses while keeping the incident schema and scorer fixed.

Workflow trace and auditability. The trace is the mechanism that makes semantic conclusions auditable. In an incident-analysis workflow, the final answer is only useful if a human can inspect why it was produced. The trace records which shard summaries were considered, which evidence objects were merged, which incident hypothesis each evidence object contributed to, and which injected incidents were missed during evaluation. The current workload records detected incidents, injected incidents, matched incident IDs, and missed incident IDs; the same trace can later be exposed through an interactive visualization layer.

Auditability matters for both correctness and operations. A false positive is explainable as a specific cluster of noisy evidence rather than as an opaque model hallucination. A false negative is linked to missing or weak evidence rather than only to a low aggregate score. The workflow orientation gives the system a natural place to attach trace information at each operator boundary.

External integration points. The prototype can integrate with external systems at three points. Upstream, data engines can supply shards, windows, or materialized feature tables. In the middle, model-serving systems can provide LLM calls for map, reduce, or final explanation operators. Downstream, incident-management or observability tools can consume structured hypotheses and evidence-linked reports. The current workload uses a synthetic generator and deterministic reducer, but the interfaces are chosen so that real Spark/Ray/Flink scans, real vector retrieval, and real serving telemetry can be substituted later.

This integration path is central to the design. The system does not duplicate mature execution engines. It makes semantic orchestration explicit across systems that otherwise remain disconnected: databases, stream processors, vector stores, LLM serving endpoints, and human-facing incident reports.

Control plane and data plane. Another way to describe the boundary is to separate the semantic control plane from the data plane. The data plane moves records, executes scans, stores logs, manages model kernels, and serves tokens. These performance-critical tasks remain in engines designed for them. The semantic control plane decides how analysis is decomposed, which summaries become evidence, which reducer combines that evidence, and how the final report exposes provenance. The prototype belongs primarily to this control plane.

This distinction is useful because LLM-native analysis often fails when the control plane is hidden inside an application script. For example, a notebook can query a database, call an LLM, and write a report, but the boundaries between data extraction, evidence selection, hypothesis formation, and explanation are implicit. Reproducing the result requires reproducing the notebook state and the human decisions around it. In our prototype, these steps are represented as workflow operators and report fields. The system can therefore record the exact reducer used, the top-k evidence budget, the seed, the matched incidents, and the missed incidents.

The control-plane view also clarifies the integration boundary. A distributed deployment may push map operators into Spark or Ray, stream events through Flink, or call an LLM endpoint for semantic reduction. In each case, the orchestration layer owns the orchestration contract: operator graph, reducer selection, evidence schema, trace capture, and report generation. This is the level at which the contribution is strongest.

Why the design is AI-native. In this workload, the prototype is AI-native in a concrete sense: the system interface treats semantic operators as first-class components. A reducer is not only a function from rows to rows; it is a component that may reason over evidence, produce a hypothesis, cite support, expose uncertainty, and preserve a trace for later inspection. The workflow is not only an execution plan; it is also a reasoning plan.

This design has consequences for implementation. The report schema carries evidence references, not only output

strings. The reducer interface is pluggable because deterministic, LLM-backed, and hybrid reducers have different cost and reliability profiles. The benchmark records false positives and false negatives in inspectable form because aggregate F1 alone does not tell an operator whether a missed incident was caused by weak evidence, poor clustering, or an overaggressive threshold. These requirements arise specifically because the workload includes LLM interpretation.

6 Workload

6.1 Scenario

The workload models telemetry from an NPU-backed LLM serving platform. Services include `prefill`, `decode`, `kv-cache`, `scheduler`, `router`, and `embedding`. Events include service, tenant, region, timestamp, latency, error flag, NPU utilization, queue depth, and generated metadata. The generator injects hidden incidents of three types:

- **Latency spike:** one service experiences elevated latency over a time window.
- **NPU saturation:** utilization increases and may affect queueing or latency.
- **Queue backlog:** scheduler or routing queues grow and create downstream symptoms.

The task is to recover injected incidents from event streams. A detected incident is useful only if it links to the supporting evidence and matches the hidden ground truth by type, service, region, and time.

Why this workload exercises the system. The workload is designed to stress the orchestration layer rather than raw storage throughput. It has enough structure to require semantic reasoning: incidents are injected across services, regions, and time windows, while local symptoms appear as latency, error, utilization, or queue-depth changes. A single shard can observe symptoms, but the global conclusion depends on how those symptoms are merged. This matches the Semantic MapReduce pattern.

The workload also forces the system to preserve evidence. If the reducer reports a latency spike, the report must include which shard-level candidates supported the decision. If it misses a saturation incident, the report should expose the missed incident ID. This is why the workload reports both detected and injected incidents, not only aggregate F1. The goal is not merely to score a classifier; it is to evaluate an auditable analysis workflow.

Finally, the workload is intentionally reproducible. Seeds, event counts, shard counts, top-k values, reducer names, and incident reports are recorded. The same matrix can be rerun when a reducer changes. This gives the prototype a benchmark harness for future LLM reducer experiments without making the current paper depend on a live model endpoint.

6.2 Pipeline and Implementation

The workload instantiates the six operators directly. Shard partitions generated telemetry by shard count, service, region, and time. MapEvidence computes shard-level statistics, anomaly candidates, and local summaries. Normalize converts candidates into scored evidence objects with provenance fields. GroupEvidence clusters candidates by incident type, service, region, and temporal overlap. SemanticReduce emits incident hypotheses with map-only, window, deterministic, or future LLM reducers. ReportTrace scores hypotheses, records matched/missed incidents, and emits per-run reports.

The workload reports precision, recall, F1, throughput, map latency, reduce latency, detected incidents, injected incidents, and missed incidents. Reports also include seed, top-k, reducer name, and incident matching metadata.

Current implementation. The current implementation has three pieces. The generator creates synthetic telemetry and hidden incidents. The map stage implements Shard, MapEvidence, and Normalize by partitioning events, computing local summaries, and producing anomaly candidates. The reducer implements GroupEvidence and SemanticReduce over shard summaries. The default reducer is deterministic. It clusters candidates using incident type, service, region, and temporal overlap. The map-only and window-aggregation reducers are diagnostic baselines that approximate alert-only and windowed-analytics workflows without incident-level semantic reduction. The `llm-stub` reducer is an interface placeholder that exercises the same resolver and report path but delegates to the deterministic reducer.

The report schema is deliberately richer than the minimum needed for precision and recall. It includes the reducer name, seed, top-k, injected incidents, detected incidents, matched incident IDs, and missed incidents. This makes the artifact useful for debugging reducer behavior. For example, when the 100K seed-7 run misses a scheduler/NPU saturation incident, the missed incident ID is preserved in the report rather than disappearing into the aggregate recall number.

Case study walkthrough. A typical run proceeds as follows. The generator emits events for services such as `prefill`, `decode`, and `scheduler`. It injects incidents such as a latency spike or NPU saturation over a bounded time window. The shard map stage computes local evidence: elevated p95 latency, higher queue depth, or high NPU utilization. Each local summary is compact enough to be passed to the reducer. The reducer then merges adjacent or related candidates into incident hypotheses. The final report presents incident type, service, region, time window, confidence-like scores, and matching information.

This walkthrough is simple, but it illustrates the value proposition. The workload is not a single SQL query and

Table 2. Reproduced workload configurations. Each configuration was run with seeds 7, 11, and 13 for map-only, window-aggregation, deterministic, and `llm-stub` reducers.

Events	Shards	Top-k	Seeds
50,000	16	12	7, 11, 13
100,000	32	16	7, 11, 13

not a single prompt. It is a workflow whose operators correspond to system responsibilities: sharding, local evidence extraction, normalization, grouping, semantic merging, reporting, and evaluation. Each responsibility can evolve independently. A future LLM reducer can replace the deterministic reducer without changing the generator or scorer; a real telemetry source can replace the synthetic generator without changing the reducer contract.

Experimental provenance. The workload is designed to make reducer behavior reproducible without placing environment-management details in the paper body. Each experiment records the workload scale, shard count, seed, reducer, top-k budget, detected incidents, missed incidents, and latency measurements. This provenance is sufficient to distinguish algorithmic changes in the reducer from changes in input generation or scoring. The accompanying artifact contains the implementation and per-run reports needed to rerun the study.

7 Evaluation

7.1 Setup

Table 2 lists the reproduced configurations. The evaluation asks three questions:

1. Does the workload exercise Shard, MapEvidence, and SemanticReduce?
2. Does incident-level reduction improve over map-only alerts and window aggregation?
3. Do the results expose failure modes that justify a stronger semantic reducer?

Comparison scope. The comparison is deliberately scoped. The paper does not report full end-to-end deployments of Spark, Flink, Ray, LangGraph, LlamaIndex, or AutoGen. Those systems have different primary responsibilities and require adapters that preserve identical evidence schemas and scoring rules. Table 3 therefore compares reducer regimes that can be run fairly inside the same workload: map-only alerting, window aggregation, and the prototype's incident-level deterministic reducer. Table 6 adds a local adapter-level comparison that fixes the reducer and scorer while changing the orchestration/product wrapper. The `llm-stub` row is an interface check rather than an LLM result.

Table 3. Aggregate reducer comparison across the six configurations. Map-only and window-aggregate are diagnostic baselines for alert-style and windowed-analytics workflows; they are not full external systems. The `llm-stub` row has identical numbers to deterministic because it currently delegates to that reducer and does not call an external LLM.

Reducer	Precision	Recall	F1
Map-only	0.1459	0.5000	0.2250
Window-aggregate	0.6432	0.9583	0.7600
Deterministic incident reducer	0.9333	0.9583	0.9392
`llm-stub`	0.9333	0.9583	0.9392

Table 4. Deterministic reducer results by workload size. Values are ordered by seeds 7, 11, and 13.

Size	Metric	Values
50K/16/12	Precision	1.00, 0.80, 0.80
50K/16/12	Recall	1.00, 1.00, 1.00
50K/16/12	F1	1.00, 0.89, 0.89
100K/32/16	Precision	1.00, 1.00, 1.00
100K/32/16	Recall	0.75, 1.00, 1.00
100K/32/16	F1	0.86, 1.00, 1.00

7.2 Reducer Quality and Failure Modes

The aggregate results in Table 3 show why incident-level reduction is the relevant operation. The map-only baseline has poor precision because it emits many duplicate shard-local candidates; it also misses incidents because the top-k budget is consumed by redundant local evidence. Window aggregation recovers most incidents, but it still emits multiple window-level detections for what should be one incident. The deterministic incident reducer improves mean F1 from 0.7600 to 0.9392 by clustering adjacent windows into incident-level hypotheses. This result is the empirical basis for the semantic-reduction argument in the current workload.

Failure modes. The per-size breakdown in Table 4 exposes two failure modes. In the 50K setting, seeds 11 and 13 detect all injected incidents but add one extra incident, reducing precision to 0.8. This indicates that threshold-style clustering can turn noisy evidence into a false incident. In the 100K setting, seed 7 detects only three of four injected incidents, missing a scheduler/NPU saturation incident and reducing recall to 0.75. This indicates that weak or distributed evidence can be filtered out before it becomes a global hypothesis.

These cases identify where a semantic reducer can add value. A reducer with access to service relationships, temporal context, and domain-specific explanations may avoid some false positives and recover some weak incidents. The current result does not show that an LLM reducer improves

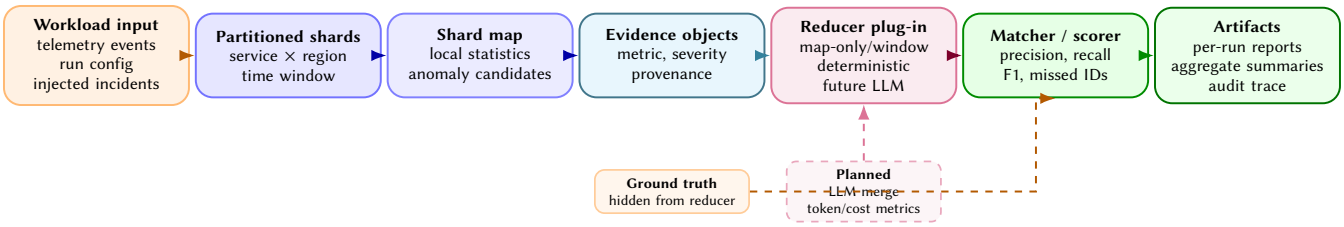

Figure 3. Evaluation harness for the large-scale analysis workload. The workload separates telemetry generation, evidence operators, reducer variants, and scoring artifacts so that map-only, window-aggregation, deterministic, LLM-backed, and hybrid reducers can be compared on identical events, injected incidents, and report schemas. Dashed components are planned future implementations rather than current experimental results.

Table 5. Local runtime metrics for the deterministic reducer. These numbers support reproducibility and regression tracking; they are not cluster-scale performance results.

Configuration	50K/16/12	100K/32/16
Throughput, events/s	63,795	63,902
Map latency, ms	96.10	196.98
Reduce latency, ms	0.37	0.34
Total latency, ms	784.11	1,564.96

quality; it establishes a controlled workload where that question can be measured.

7.3 Runtime and Adapter Overhead

The local runtime metrics in Table 5 show that the map stage dominates measured workload time because it scans and summarizes event shards. The reduce stage is sub-millisecond in the current baseline because it performs deterministic clustering over compact candidate summaries. The workload therefore exercises the intended architecture: raw event processing precedes semantic reduction, and the reducer sees compressed evidence rather than all raw events.

These numbers are not distributed systems performance claims. The current runs are local and do not evaluate cluster scheduling, shuffle, or fault recovery. They establish a reproducible baseline for later reducer studies. If an LLM reducer improves recall but adds latency or token cost, the same harness can quantify the tradeoff. A complete evaluation would report end-to-end latency and cost for deterministic, LLM-backed, and hybrid reducers under the same workload.

Adapter-level comparison. The adapter-level run in Table 6 answers a different question from Table 3. It does not compare semantic algorithms. Instead, it asks whether the workload can be driven through adjacent orchestration products while preserving the same evidence contract and scorer. All rows produce the same F1 because the deterministic reducer is fixed. The measured signal is integration overhead and artifact compatibility. LangGraph and LlamaIndex-core wrappers preserve the workload contract with similar local throughput in this small run; Ray local execution is slower here because shard event lists are serialized into local Ray tasks and the host reported Ray runtime pressure. Tuned Ray, Spark, and Flink adapters are outside the current experiment.

Table 6. Local adapter-level comparison with the deterministic incident reducer fixed. Quality is identical by construction; the table measures wrapper and evidence-packaging overhead, not semantic superiority. Ray ran in local mode on a host that emitted NPU detection and `/tmp/ray` space warnings, so its row is not tuned cluster performance.

Adapter	F1	Throughput	Total ms
Prototype	0.9392	58,594	1,301.33
LangGraph local	0.9392	60,051	1,257.80
LlamaIndex docstore	0.9392	61,519	1,230.84
Ray local	0.9392	39,739	1,870.75

Table 7. Single-NPU online endpoint run. The endpoint served Qwen2.5-7B on one Ascend 910B2 NPU. Each row reports eight measured streaming requests after warmup. The table validates endpoint integration and measurement plumbing rather than SOTA serving performance.

Concurrency	OK	TTFT ms	TPOT ms	Tok/s
1	8/8	117.69	70.18	13.82
2	8/8	1,494.64	70.24	13.883

7.4 Real-Online Readiness

The online endpoint results in Table 7 attach a live model-serving endpoint to the workload effort. The purpose is not to benchmark serving throughput, but to validate that the reducer path can be connected to an external LLM service

and measured with standard serving metrics. The concurrency-2 row has much larger TTFT because requests queue behind the active decode in this constrained setup. The similar TPOT values indicate that the decode path itself is stable across the two small runs.

Audit artifacts. The benchmark produces artifacts beyond a single summary table. Each run records the configuration, reducer name, injected incidents, detected incidents, matched incident IDs, missed incident IDs, and latency measurements. This matters because reducer experiments require more than a single score. A researcher should be able to identify which incident changed and inspect the evidence path that caused the change.

The artifact structure also supports regression testing. If a code change improves mean F1 but introduces a systematic false positive in one service or region, the per-run reports expose that pattern. If an LLM reducer improves recall only by emitting many vague incidents, precision falls and the detected incident list shows why. If an LLM reducer is too expensive, latency and token accounting can be attached to the same report schema. In this sense, the workload is not only a synthetic generator; it is an experimental harness for studying semantic reducers under controlled conditions.

7.5 Result Interpretation

Precision, recall, and F1 have a different meaning in this setting than in a conventional classifier benchmark. Precision measures whether the reducer avoids inventing incidents from noisy local evidence. Recall measures whether the reducer preserves weak but real incidents through evidence compression and global merging. F1 summarizes the tradeoff, but the more important signal is the error type. A false positive suggests that the reducer accepted an insufficient evidence cluster. A false negative suggests that the map stage failed to surface enough evidence, or that the reducer failed to connect distributed symptoms.

The 50K and 100K configurations show both sides of the tradeoff. At 50K, recall is perfect across seeds, but two seeds have precision 0.8 because an extra incident is emitted. At 100K, precision is perfect across seeds, but one seed misses an incident. These cases make reducer design decisions observable. An LLM-backed reducer is valuable only if it changes these error patterns while preserving evidence links and keeping latency and cost within bounds.

What the results say about the prototype. The results support a modest but important conclusion: the workflow boundary can run a nontrivial Semantic MapReduce workload end to end. The system instantiates the full operator vocabulary: it shards partitioned data, maps local evidence, normalizes and groups evidence, reduces candidates into incidents, scores reports against ground truth, and preserves missed-incident metadata. These capabilities form the systems substrate for evaluating LLM-backed reducers.

The results also show why the prototype exposes reducer plug-ins rather than hard-coding a single strategy. In 50K runs, the deterministic reducer finds all ground-truth incidents but sometimes adds an extra incident. In the 100K seed-7 run, it avoids false positives but misses a scheduler/NPU saturation incident. These are different failure modes. Improving one may hurt the other. The reducer interface makes this tradeoff measurable because alternative reducers can be compared under the same map outputs, evidence schema, and scoring logic.

Implications for a real LLM reducer. The deterministic baseline provides a lower bound on system complexity. It shows that many incidents can be recovered without an LLM when the evidence is clean and the clustering rules match the incident structure. An LLM reducer has a clear role when reduction requires reconciling partial evidence, interpreting service relationships, explaining why two symptoms belong together, or turning structured hypotheses into operator-facing narratives.

This motivates a hybrid design. Deterministic map stages can scan large event volumes and produce compact candidate evidence. A deterministic pre-reducer can remove obvious duplicates. The LLM reducer can then operate over a bounded set of evidence objects rather than over raw telemetry. This placement respects latency, cost, and safety constraints. It also gives the evaluation a fair comparison: the LLM is not rewarded for doing basic scanning work that data engines already do well; it is evaluated on semantic reduction and explanation.

8 Discussion

Why the prototype is more than glue code. A natural skepticism is that the prototype is only glue code around data systems and LLMs. The workload suggests a sharper distinction. Glue code can call a database, invoke a model, and format a response. The prototype provides a reusable execution structure for semantic analysis: operators, streams, reducers, evidence objects, reports, and traces. These pieces are system concepts rather than application-specific callbacks.

This matters because LLM-native analysis uses multiple reducer regimes. Some reducers are deterministic because they are cheap and reproducible. Some reducers use LLMs because they need domain language, causal reasoning, or flexible explanation. Others are hybrid: deterministic filters first, LLM-based adjudication second. Without a workflow/runtime layer, each variant becomes a separate application. With the prototype, they become plug-ins under a common evidence and scoring contract.

The prototype is also stronger than a single-agent design for this workload. Agents are useful for open-ended exploration, but large-scale analysis needs predictable partitioning, bounded context, reproducible intermediate artifacts, and audit trails. Semantic MapReduce gives the system those

properties while still leaving room for LLM calls inside map, reduce, or final explanation operators.

What the baseline shows. The deterministic baseline is intentionally conservative. It demonstrates that shard-level evidence extraction and incident-level reduction can be implemented as a reproducible workflow. It also shows that the problem is not trivial: varying seeds produces both false positives and false negatives. This makes the workload useful for comparing reducer designs.

Equally important, the baseline shows where the prototype remains modular. The map stage and report schema are not tied to the deterministic reducer. The same run metadata can compare deterministic and LLM-backed reducers. The same missed-incident field can diagnose different reducer failures. The same evaluation harness can produce per-run reports, aggregate summaries, and paper-level tables. This infrastructure turns semantic reduction into an evaluable systems question.

What it does not show. The current results do not show that the prototype is a production distributed execution engine. They do not show that LLM reduction improves precision or recall, because no real LLM reducer has been evaluated yet. They also do not constitute full end-to-end comparisons against Spark, Flink, Ray, LangGraph, LlamaIndex, or AutoGen-style systems. The baselines isolate reducer behavior under a shared workload harness, and the synthetic telemetry does not fully capture real accelerator-backed serving operations.

Next minimal system step. The immediate research agenda is to implement a pluggable LLM semantic reducer. The reducer consumes the same evidence objects as the deterministic baseline and produces the same incident schema. The corresponding experiment compares deterministic, LLM, and hybrid reducers on identical seeds and ground truth, reporting precision, recall, F1, evidence coverage, token count, model latency, monetary cost, and human analyst agreement.

The second agenda item is to connect the workload to a real upstream engine or trace source. A Spark or Ray job could produce the same evidence objects from larger telemetry tables; a real serving deployment could provide logs and accelerator metrics; a vector store could provide related textual evidence. These integrations would not change the prototype's role. They would strengthen the evidence that the prototype is the orchestration layer connecting execution engines, serving systems, semantic reducers, and auditable reports.

Toward a full system evaluation. A full system evaluation has four axes beyond the current prototype. The first axis is reducer quality: deterministic, LLM, and hybrid reducers are compared on identical raw events and ground truth. The second axis is orchestration overhead: the prototype reports how much latency it adds beyond upstream data processing and downstream model serving. The third axis is evidence efficiency: the system measures how much raw telemetry is compressed into evidence objects, how much evidence is sent to the reducer, and how much context a model consumes. The fourth axis is audit usefulness: human evaluators judge whether reports contain enough evidence to validate or reject an incident hypothesis.

These axes align with the orchestration-layer thesis. If the prototype is only a wrapper, it will be hard to show benefits beyond application convenience. If the prototype is a real orchestration layer, reducer substitution, cost accounting, trace inspection, and integration with external engines become measurable system properties. The current workload establishes the skeleton for that evaluation; a complete system study replaces the stub with a real LLM reducer and connects the input side to real or replayed telemetry.

9 Related Work

Large-scale data processing. MapReduce, Spark, and Flink provide the execution substrate for large-scale batch and stream analytics [1, 2, 8]. Semantic MapReduce borrows the local/global decomposition but changes the reducer contract from fixed aggregation to semantic evidence fusion.

Distributed AI execution. Ray provides a general distributed runtime for AI and Python applications [6]. The prototype can use such systems as execution substrates, but its focus is the explicit orchestration of LLM reasoning workflows and evidence traces.

LLM serving. vLLM shows how memory management and scheduling shape LLM serving performance [3]. Our workload is motivated by serving telemetry in accelerator-backed deployments. This paper does not modify the serving engine; serving systems act as external sources of telemetry and possible model endpoints for semantic reducers.

LLM application frameworks. LangGraph, LlamaIndex, and AutoGen provide useful abstractions for agents, retrieval, tool use, and multi-agent workflows [4, 5, 7]. A full comparison requires verifying their dataflow, streaming, runtime, and audit semantics under the same workload. The narrower comparison here is that the prototype emphasizes workflow/stream/runtime orchestration with explicit semantic reduction and evidence provenance.

Data+AI platforms. Systems such as Databricks, Snowflake Cortex, and BigQuery ML integrate analytics engines with machine learning and LLM functionality. These platforms are important reference points, but they require systematic comparison before strong claims about relative capability. The prototype differs in emphasis: it is an open workflow/runtime layer that can sit above multiple engines and make reducer behavior auditable. The main comparison dimensions are APIs, execution boundaries, governance mechanisms, and traceability features.

Observability and incident management. Production observability systems collect metrics, logs, traces, alerts, and incident timelines. They provide the raw material for the workload studied here. The prototype does not replace them. Instead, Semantic MapReduce asks how evidence from these systems can be converted into auditable incident hypotheses. This is an important distinction: an alert can say that a threshold fired, while a semantic reducer should explain how multiple alerts and metrics combine into one operational hypothesis.

Agentic data analysis. Recent agentic systems can write queries, call tools, inspect results, and refine an answer. Such systems are useful for exploratory analysis, but their control flow may be difficult to reproduce when applied to high-volume operational data. The prototype chooses a more constrained structure for this paper: explicit map stages, explicit reducer stages, and explicit evidence objects. This reduces flexibility, but it improves replayability, evaluation, and auditability.

Positioning summary. The closest intellectual ancestor of Semantic MapReduce is MapReduce itself: local processing followed by global reduction. The closest practical neighbors are stream processors, distributed AI runtimes, LLM workflow frameworks, RAG/data frameworks, and data+AI platforms. The prototype sits between these categories. It is not the storage engine, not the model server, and not merely an agent framework. It is the orchestration layer that makes semantic data analysis explicit enough to evaluate.

This positioning is intentionally cautious. The paper does not claim that the prototype is more scalable than Spark or Ray, more feature-complete than data+AI platforms, or more general than agent frameworks. The sharper claim is that the prototype gives this workload the right system structure: standard evidence operators, semantic reduction, evidence-linked reporting, and workflow traces under one reproducible contract.

10 Limitations

The current study has three main limitations. First, the workload is synthetic: it is modeled after accelerator-backed LLM serving telemetry, but it is not yet a replay of production traces. Second, the evaluated semantic reducer is deterministic; the `llm-stub` row exercises the interface but is not an LLM result. Third, the experiments are local and do not evaluate distributed shuffle, exactly-once processing, fault recovery, or full end-to-end deployments of external systems.

These boundaries define the claim. The results support Semantic MapReduce as an orchestration abstraction and show that the prototype can coordinate shard-level analysis, evidence objects, incident reduction, and workflow traces under a reproducible contract. They do not show that the prototype replaces data engines or model-serving systems, nor that LLM reduction already improves incident recovery. The next step is to evaluate deterministic, LLM-backed, and hybrid reducers on real or replayed telemetry under the same evidence and scoring interface.

11 Conclusion

Large-scale LLM-augmented analysis needs more than fast scans and more than chat over data. It needs a workflow layer that can partition observations, compress evidence, semantically reduce hypotheses, and preserve audit trails. Semantic MapReduce captures this need by extending the map/reduce structure from numeric aggregation to evidence fusion and explanation. The prototype's dataflow, stream, runtime, and serving boundaries make it a concrete substrate for this direction. The current workload demonstrates shard-level analysis and incident reduction while identifying the next experimental step: LLM-backed and hybrid reducers evaluated against the same evidence and scoring contract.

References

[1] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. 2015. Apache Flink: Stream and Batch Processing in a Single Engine. *IEEE Data Engineering Bulletin* (2015).

[2] Jeffrey Dean and Sanjay Ghemawat. 2004. MapReduce: Simplified Data Processing on Large Clusters. In *Proceedings of the 6th Symposium on Operating Systems Design and Implementation.*

[3] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles.*

[4] LangChain. 2026. LangGraph: Build Stateful, Multi-Actor Applications with LLMs. https://github.com/langchain-ai/langgraph. Accessed 2026-06-30.

[5] LlamaIndex. 2026. LlamaIndex: Data Framework for LLM Applications. https://github.com/run-llama/llama_index. Accessed 2026-06-30.

[6] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. 2018. Ray: A Distributed Framework for Emerging AI Applications. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation.*

[7] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed H. Awadallah, Adam White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155 [cs.AI]

[8] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. 2012. Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. In *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation.*